

Manual técnico

Simple Stock Flow — sistema de productos y ventas

Autor: Jesus Ariel Gonzalez Bonilla

Versión: 1.1

Fecha: 21 de septiembre de 2026

Dirigido a: quien despliega, opera, mantiene o extiende el sistema

Índice

1. Sobre este manual.....	5
1.1 A quién va dirigido.....	5
1.2 Qué encontrará aquí y qué no.....	5
1.3 Convenciones.....	5
2. El sistema de un vistazo.....	7
2.1 Qué hace Simple Stock Flow.....	7
2.2 Las tres unidades desplegables.....	7
2.3 Dónde vive cada cosa en disco.....	8
2.4 Puertos y direcciones.....	8
3. Requisitos previos.....	9
3.1 Para desplegar hace falta Docker y nada más.....	9
3.2 Para desarrollar en local.....	9
3.3 Entorno verificado.....	9
4. Instalación y primer arranque.....	11
4.1 Obtener los repositorios.....	11
4.2 Configurar el archivo de entorno.....	11
4.3 Levantar el sistema.....	12
4.3.1 Despliegue normal.....	12
4.3.2 Despliegue de desarrollo.....	12
4.4 Qué ocurre durante el arranque.....	12
4.5 Comprobar que quedó bien.....	12
4.6 Sembrar datos de demostración.....	14
5. Configuración.....	16
5.1 Las claves del archivo de entorno.....	16
5.2 Cómo llegan al servicio.....	16
5.3 Encender y apagar la documentación ejecutable.....	17
5.4 Cambiar el puerto publicado.....	17
6. Operación.....	18
6.1 Arrancar, parar y reiniciar.....	18
6.2 Ver el estado y la salud.....	18
6.3 Leer los registros.....	18
6.4 Inspeccionar la base de datos.....	19
6.4.1 Qué garantiza el esquema.....	19
6.5 Respaldo.....	19
6.5.1 La base de datos.....	19
6.5.2 El volumen de imágenes.....	19
6.6 Restauración.....	20
6.7 Desplegar una versión nueva del código.....	20

7. La interfaz de programación.....	21
7.1 Cómo se obtiene un testigo.....	21
7.2 Inventario de operaciones.....	21
7.3 Swagger como consola de pruebas.....	22
7.4 Qué significa cada código de respuesta.....	22
7.5 La operación de venta, paso a paso.....	23
7.6 La ruta pública de imágenes.....	23
7.7 El camino que recorre una petición.....	24
8. Mantenimiento del código.....	26
8.1 Compilar y probar el servicio.....	26
8.2 Compilar y probar el portal.....	26
8.3 La regla que no se puede romper.....	26
8.4 Dónde va cada cosa.....	27
8.5 Añadir un caso de uso.....	28
8.6 Añadir una migración.....	28
9. Diagnóstico de problemas.....	30
9.1 Síntomas frecuentes.....	30
9.2 Cuando el documento y el sistema discrepan.....	30
10. Anexo: comandos de referencia rápida.....	32
11. Documentos relacionados.....	33

Figuras

Figura 1. Topología desplegada. Las líneas discontinuas solo existen con el solapamiento de desarrollo....	7
Figura 2. Pantalla de acceso del portal: si aparece, el despliegue terminó bien y el intermediario alcanza al servicio.....	13
Figura 3. El catálogo del portal después de sembrar los datos de demostración.....	15
Figura 4. El registro de una venta: qué decide el dominio, qué congela la línea y cuándo se reintenta.....	23
Figura 5. Camino completo de una petición, de la pantalla al motor y de vuelta.....	24
Figura 6. Referencias reales entre los proyectos de la solución.....	27
Figura 7. Estructura interna de los repositorios, con el sembrador ya fuera de la infraestructura.....	27
Figura 8. Las mismas tres capas a los dos lados, y el único punto donde se tocan.....	28

Tablas

Tabla 1. Qué no está en este manual y dónde está.....	33
Tabla 2. Qué hace cada unidad.....	7
Tabla 3. Direcciones del sistema.....	8
Tabla 4. Versiones del entorno de comprobación.....	9
Tabla 5. Claves declaradas en .env.example.....	16
Tabla 6. De la clave del entorno a la configuración del servicio.....	16
Tabla 7. Campos útiles de cada línea de registro.....	18
Tabla 8. Las quince operaciones y su autorización.....	21
Tabla 9. Códigos de respuesta y su causa.....	22
Tabla 10. Qué hacer cuando algo falla.....	30
Tabla 11. Referencia rápida.....	32
Tabla 12. Dónde seguir leyendo.....	33

1. Sobre este manual

1.1 A quién va dirigido

Este manual es para quien tenga que **poner el sistema en marcha, mantenerlo vivo o modificarlo**: personal de despliegue, de operación y de desarrollo. Supone soltura con una línea de órdenes, con contenedores y con HTTP. No supone ningún conocimiento previo de Simple Stock Flow.

Si lo que busca es cómo **usar** el sistema desde su interfaz —crear un producto, registrar una venta, sacar el reporte—, ese es el manual de usuario y este no lo cubre.

1.2 Qué encontrará aquí y qué no

Aquí están los **procedimientos**: instalar, configurar, arrancar, verificar, respaldar, actualizar, diagnosticar y extender. Cada comando de este manual se ejecutó contra el sistema real antes de escribirlo, en las fechas que dice la nota del apartado siguiente; los pocos que no se pudieron ejecutar sin destruir datos están marcados como tales.

Tres cosas quedan deliberadamente fuera, y conviene saber dónde están:

Tabla 1. Qué no está en este manual y dónde está

Lo que busca	Dónde vive
Cómo se usa el sistema desde la pantalla	El manual de usuario, que acompaña a este documento
El cuerpo exacto de cada petición y cada respuesta	La página de documentación ejecutable, en /swagger. Se genera del propio código y no puede desfasarse
Por qué la arquitectura es como es, y qué cuesta cada garantía	El documento de arquitectura, que acompaña a este manual

La segunda fila importa más de lo que parece: una descripción de la interfaz copiada a mano en un manual envejece, y la que se genera del código no.

1.3 Convenciones

- Las **órdenes de terminal** van en bloques con fondo gris. Se escriben para bash; en Windows funcionan tal cual desde Git Bash.
- Los **nombres de archivo, variables y rutas** van en fuente monoespaciada: `docker-compose.yml`, `JWT_SIGNING_KEY`, `/api/products`.
- Los **avisos** se distinguen por color: azul informa, ámbar advierte de algo que suele costar tiempo, rojo señala una acción que puede destruir datos.

El sistema mantiene además una **frontera idiomática** que explica por qué en la misma pantalla conviven dos idiomas: lo que ve quien **usa** el sistema está en español —rutas, mensajes, interfaz— y lo que ve

quien lo **mantiene** está en inglés —identificadores, comentarios y registros de ejecución—. No es un descuido.

Nota. Las comprobaciones de este manual se hicieron el **20 y el 21 de septiembre de 2026** contra el sistema levantado en la máquina de desarrollo. Los git clone del capítulo 4 son del 21: hasta ese día ningún repositorio tenía remoto ni confirmación, y los seis la recibieron entonces. Todo lo demás es del 20. Las versiones exactas están en el capítulo 3.

2. El sistema de un vistazo

2.1 Qué hace Simple Stock Flow

Simple Stock Flow gestiona un **catálogo de productos** y el **registro de ventas** sobre ese catálogo, con un reporte agregado por rango de fechas y control de acceso por rol. Se despliega como tres contenedores y publica **un solo puerto** al exterior.

2.2 Las tres unidades desplegables

El navegador nunca habla con la interfaz de programación ni con la base de datos: habla con el portal, y el portal reenvía. Esa es la primera cosa que hay que tener clara, porque explica casi todo lo demás.

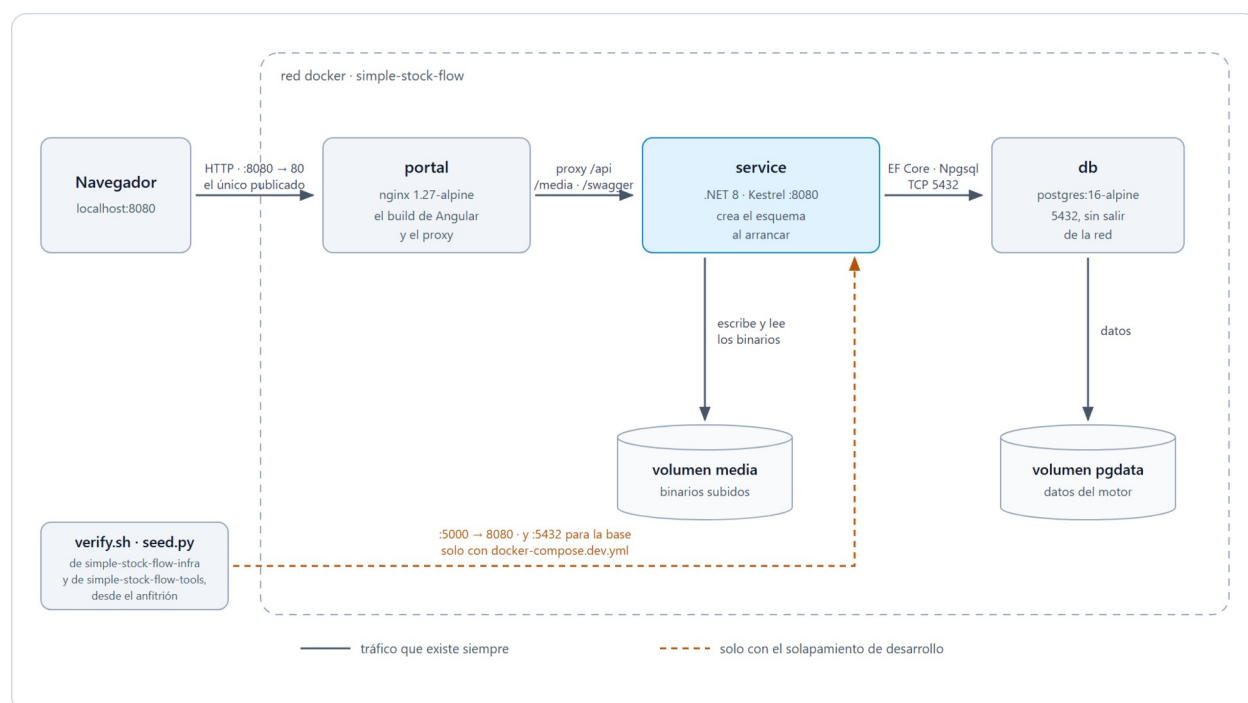


Figura 1. Topología desplegada. Las líneas discontinuas solo existen con el solapamiento de desarrollo

Tabla 2. Qué hace cada unidad

Unidad	Responsable de	No responsable de
simple-stock-flow-infra	Contenedores, red, volúmenes, comprobaciones de salud y variables de entorno	El esquema de datos: no contiene una sola sentencia SQL de definición
simple-stock-flow-api	Reglas de negocio, interfaz de programación y el esquema completo , que aplica al arrancar	La orquestación del despliegue
simple-stock-flow-portal	Interfaz de usuario y sus propios casos de uso	El almacenamiento: solo habla con la interfaz de programación

2.3 Dónde vive cada cosa en disco

Los directorios son **hermanos, y tienen que serlo**: el archivo de composición construye las imágenes desde `../simple-stock-flow-api` y `../simple-stock-flow-portal`. Anidar uno dentro de otro, o clonarlos por separado, deja la construcción sin contexto.

2.4 Puertos y direcciones

Tabla 3. Direcciones del sistema

Dirección	Disponible	Qué sirve
<code>http://localhost:8080</code>	Siempre	El portal. Es el único puerto publicado en un despliegue normal
<code>http://localhost:8080/swagger</code>	Siempre	La documentación ejecutable de la interfaz de programación, a través del portal. Desde la barra del portal hay un enlace directo , que la abre en una pestaña nueva
<code>http://localhost:5000</code>	Solo en desarrollo	La interfaz de programación directa, sin pasar por el portal
<code>http://localhost:5000/health</code>	Solo en desarrollo	Señal de vida del proceso. El portal no la reenvía
<code>localhost:5432</code>	Solo en desarrollo	El motor PostgreSQL, para inspeccionarlo con un cliente

El puerto del portal se cambia con `PORTAL_PORT`; los dos de desarrollo, con `SERVICE_PORT` y `POSTGRES_PORT`.

3. Requisitos previos

3.1 Para desplegar hace falta Docker y nada más

Un despliegue completo **no necesita .NET, ni Node, ni un cliente de base de datos** instalados en la máquina. Todo se construye y se ejecuta dentro de contenedores. Basta con Docker y su complemento de composición.

3.2 Para desarrollar en local

Si va a compilar o ejecutar pruebas fuera de los contenedores, necesita además el kit de .NET 8 y Node 22 o superior. Python solo hace falta para el sembrador de datos de demostración, que además necesita Docker: alcanza la base con `docker compose exec` para repartir las ventas en el tiempo.

Atención. La solución fija `RollForward=LatestPatch`, de modo que **exige el tiempo de ejecución de .NET 8 aunque tenga instalada una versión mayor**. Con solo .NET 10 instalado, `dotnet test` falla sin decir claramente por qué.

Cuidado. El usuario de la base **necesita permiso para crear extensiones**. Una de las migraciones instala `pg_trgm`, que es lo que sostiene el índice de búsqueda por nombre de producto. Con un usuario sin ese permiso, el servicio arranca, intenta migrar y falla. En el despliegue con el `compose` no es problema porque el usuario que crea el contenedor es el dueño de la base; contra un motor administrado, compruébelo antes.

3.3 Entorno verificado

Estas son las versiones con las que se comprobó este manual. No son mínimos estrictos salvo donde se indica.

Tabla 4. Versiones del entorno de comprobación

Herramienta	Versión	Necesaria para
Docker	29.4.2	Todo. Es el único requisito real
Docker Compose	v5.1.3	Levantar el conjunto
.NET SDK	10.0.401	Compilar el servicio en local
.NET, tiempo de ejecución	8.0.31 y 10.0.12	Ejecutar las pruebas: el 8 es obligatorio
Node.js	v24.11.1	Compilar y probar el portal en local
npm	11.6.2	Instalar las dependencias del portal
Python	3.11.9	Solo para el sembrador de <code>simple-stock-flow-tools</code> , que además usa <code>docker compose exec</code>

Herramienta	Versión	Necesaria para
dotnet-ef	10.0.11	Solo para crear migraciones nuevas

4. Instalación y primer arranque

4.1 Obtener los repositorios

Coloque los directorios **al mismo nivel** dentro de una carpeta contenedora. La disposición no es cosmética: forma parte de lo que la construcción da por supuesto.

```
una-carpeta-cualquiera/  
├── simple-stock-flow-infra/    ← desde aquí se levanta todo  
├── simple-stock-flow-api/  
├── simple-stock-flow-portal/  
├── simple-stock-flow-docs/    ← no participa en la construcción  
├── simple-stock-flow-tools/   ← opcional: los datos de demostración  
└── simple-stock-flow-page/    ← opcional: la página pública
```

Los tres primeros son los únicos que hacen falta para levantar el sistema. `simple-stock-flow-tools` guarda el sembrador y las veintidós imágenes de demostración, y vive aparte por una razón concreta: exige **Python y la biblioteca de peticiones web** además del motor de contenedores —que usa para repartir las ventas en el tiempo—, mientras que la infraestructura promete que para levantar el sistema basta con el motor de contenedores. Sus archivos están en la **raíz** de ese repositorio.

Los tres primeros se obtienen con `git clone`. Están publicados en la organización `code-dev-projects`, y no hay submódulos a propósito, para que cada repositorio se pueda clonar y leer solo:

```
mkdir simple-stock-flow && cd simple-stock-flow  
git clone https://github.com/code-dev-projects/simple-stock-flow-infra.git  
git clone https://github.com/code-dev-projects/simple-stock-flow-api.git  
git clone https://github.com/code-dev-projects/simple-stock-flow-portal.git
```

Nota. `simple-stock-flow-tools` y `simple-stock-flow-page` se clonan igual y desde la misma organización, pero solo si los necesita: el primero siembra los datos de demostración y el segundo es la página pública. `simple-stock-flow-docs` es **privado** —guarda el expediente del proyecto— y no hace falta para levantar nada.

4.2 Configurar el archivo de entorno

1. Sitúese en `simple-stock-flow-infra`, que es el directorio desde el que se opera todo.
2. Copie la plantilla: `cp .env.example .env`. La plantilla declara **todas** las claves y **ningún** valor secreto.
3. La plantilla trae **cuatro** valores vacíos. Rellene los tres que hacen falta para arrancar: `POSTGRES_PASSWORD`, `JWT_SIGNING_KEY` —de al menos 32 caracteres— y `ADMIN_PASSWORD`.
4. El cuarto, `DEMO_SELLER_PASSWORD`, solo hace falta si va a sembrar los datos de demostración: sin él el sistema arranca igual y lo que se detiene es el sembrador.

Atención. `JWT_SIGNING_KEY` **no tiene valor por omisión y no lo tendrá nunca**. Si falta o mide menos de

32 caracteres, la interfaz de programación se niega a arrancar y dice por qué. Es deliberado: arrancar con una clave de relleno repartiría credenciales falsificables y el fallo aparecería lejos de su causa.

4.3 Levantar el sistema

De aquí en adelante los comandos se teclean **desde simple-stock-flow-infra**, que es donde le dejó el apartado anterior; cuando alguno haya que ejecutarlo en otro directorio, el propio bloque empieza con el `cd` que lleva hasta él.

4.3.1 Despliegue normal

Use únicamente el archivo base. Publica el portal y deja dentro de la red la interfaz de programación y la base.

```
docker compose up -d --wait
```

4.3.2 Despliegue de desarrollo

Añada siempre el solapamiento. Publica además la interfaz de programación en `:5000` y la base en `:5432`, que es lo que necesitan el servidor de desarrollo del portal, el guion de verificación y el sembrador.

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d --wait
```

Atención. No mezcle los dos arranques. Levantar con el archivo base una pila que ya corría con el solapamiento **recrea el contenedor del servicio y le retira el puerto publicado, sin avisar**. Si `:5000` deja de responder de golpe, es esto.

4.4 Qué ocurre durante el arranque

El orden no lo marca una lista de precedencias sino **comprobaciones de salud**: cada contenedor espera a que el anterior responda, no a que exista. Por eso no hay un primer arranque que falla y un segundo que funciona.

El motor llega **vacío**: el contenedor de PostgreSQL crea la base y el usuario, y nada más. El esquema, sus restricciones y las categorías de referencia los aplica el propio servicio con sus migraciones. Un guion de inicialización en el contenedor del motor no serviría, porque solo se ejecuta con el volumen vacío y la segunda versión del esquema no se aplicaría jamás.

Una de esas migraciones hace algo más que crear tablas: **instala la extensión `pg_trgm`** y crea con ella un índice de trigramas sobre el nombre del producto, además de un índice parcial que solo cubre los productos activos. Es la razón del permiso que pide el aviso del capítulo anterior.

4.5 Comprobar que quedó bien

1. Compruebe que los tres contenedores están sanos: `docker compose ps`. La columna de estado debe decir `healthy` en `db`, `service` y `portal`.
2. Abra `http://localhost:8080`. Debe aparecer la pantalla de acceso.
3. Abra `http://localhost:8080/swagger`. Debe aparecer la documentación ejecutable con sus operaciones.
4. Con el solapamiento de desarrollo, compruebe la señal de vida: `curl http://localhost:5000/health` responde 200 con `{"status": "ok"}`.

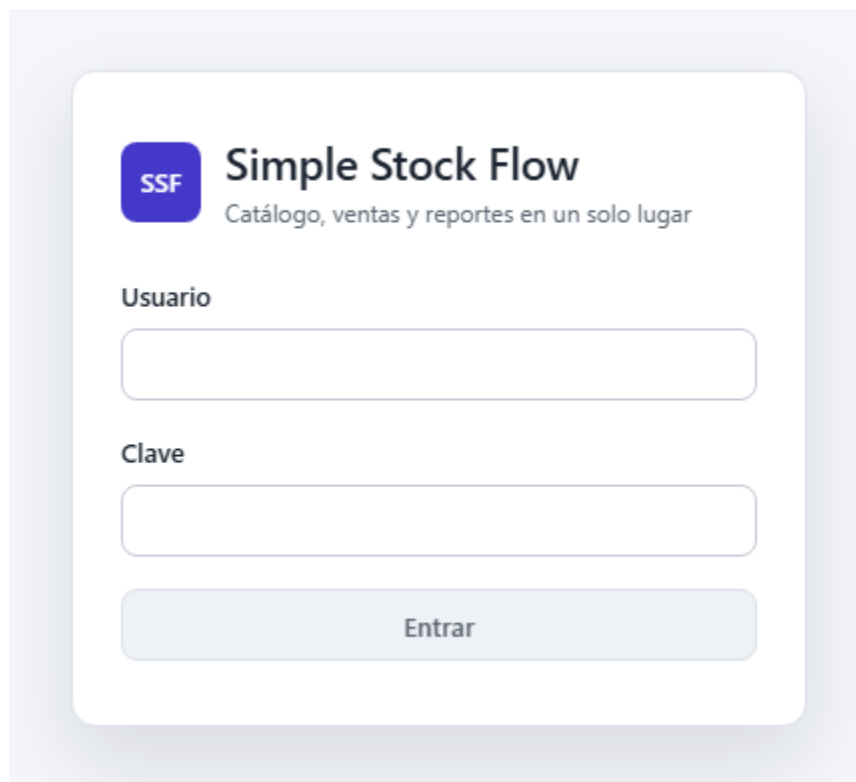


Figura 2. Pantalla de acceso del portal: si aparece, el despliegue terminó bien y el intermediario alcanza al servicio

Para una comprobación exhaustiva existe un guion que recorre el sistema entero —el portal, la autorización de la interfaz de programación, la persistencia tras un reinicio, la negativa a arrancar sin clave de firma, las imágenes a través del proxy y las dos rutas de la documentación—:

Termina imprimiendo cuántas pasaron y cuántas fallaron, y esa cifra es la que vale: este manual no la escribe, porque una cifra en prosa envejece sin que nadie se entere. Entre ellas hay dos que conviene conocer porque vigilan cosas que ningún test puede ver: la §6c comprueba que **todas** las líneas de registro de petición llevan su identificador de correlación, y la §6d cuenta **binarios huérfanos** en el volumen de imágenes —archivos que ya no pertenecen a ningún producto y que, por la ruta pública, seguirían sirviéndose para siempre—.

Nota. Las secciones §8 y §9 no auditan el sistema sino los **documentos del proyecto**, que viven en `simple-stock-flow-docs`, un repositorio **privado**. Con solo los tres repositorios públicos clonados, esas

comprobaciones fallan diciendo qué archivo les falta y el guion sale con código distinto de cero. Es lo esperado: lo que audita el sistema está en las secciones anteriores.

```
./verify.sh          # deja la pila levantada
./verify.sh --down   # la apaga al terminar
```

Atención. `verify.sh` **reconstruye las imágenes** antes de medir, y eso tarda. Es deliberado: adoptar la imagen que ya lleva la etiqueta deja servir un paquete anterior mientras todas las comprobaciones siguen en verde, que es exactamente el fallo que el guion existe para impedir.

4.6 Sembrar datos de demostración

El sembrador crea un catálogo con imágenes, una cuenta de vendedor y un historial de ventas repartido en el tiempo. Entra **por la interfaz de programación y no por SQL**, de modo que los datos de demostración obedecen las mismas reglas que lo que teclea una persona.

Hay **una sola excepción, y está declarada**: el instante de cada venta se reparte con un update aislado — `spread_sales_over_time`—, lanzado con `docker compose exec` y **después** de que la venta ya pasó por todas las reglas. Se hace así porque una venta se sella con la hora en que se registra, de modo que un sistema recién sembrado tendría todas sus ventas dentro del mismo minuto y el reporte por rango de fechas no tendría nada que enseñar. Es también la razón por la que `seed.py` necesita Docker además de Python.

```
cd ../simple-stock-flow-tools
pip install requests    # solo la primera vez
python seed.py
```

`requests` es la única biblioteca que el sembrador necesita: lo demás lo pone el motor de contenedores. Las imágenes no se generan aquí —vienen hechas en `img/`—, y el guion que las dibuja, `make_pictures.py`, es el que pide Pillow y un navegador.

Nota. Necesita el solapamiento de desarrollo levantado: el sembrador habla con `http://localhost:5000`. Lee las credenciales del administrador del archivo `.env` de `simple-stock-flow-infra` —por eso los dos directorios tienen que estar uno al lado del otro— y no las imprime nunca.

Al terminar, el catálogo del portal queda como en la figura siguiente. Es la comprobación más barata de que las tres unidades se entienden: la pantalla la sirve el portal, las filas las sirve la interfaz de programación y las imágenes salen del volumen de binarios.

 Simple Stock Flow

[Productos](#)
[Nueva venta](#)
[Ventas](#)
[Reportes](#)
[Nuevo vendedor](#)

Carrito 0

admin
Administrador
 [Salir](#)

Productos

23 productos en el catálogo

[Nuevo producto](#)

NOMBRE	CATEGORÍA	PRECIO	STOCK	ACCIONES
 Alicata universal 8"	Herramientas	\$ 32.400	20	Editar
 Bombilla LED 9 W luz cálida	Electricidad	\$ 9.800	80	Editar
 Brocha 3" de cerda natural	Pinturas	\$ 12.800	32	Editar
 Cable encauchetado 3x14, por metro	Electricidad	\$ 6.400	235	Editar
 Candado de seguridad 50 mm	General	\$ 28.700	24	Editar
 Cinta de teflón 12 m	Fontanería	\$ 2.900	144	Editar
 Escalera tijera 5 pasos en aluminio	General	\$ 219.000	0	Editar
 Esmalte sintético negro 1/4	Pinturas	\$ 34.600	11	Editar
 Extensión múltiple 6 tomas	Electricidad	\$ 46.900	11	Editar
 Flexómetro 5 m	Herramientas	\$ 18.500	36	Editar

Figura 3. El catálogo del portal después de sembrar los datos de demostración

5. Configuración

5.1 Las claves del archivo de entorno

Tabla 5. Claves declaradas en `.env.example`

Clave	¿Obligatoria?	Para qué sirve
POSTGRES_DB	Sí	Nombre de la base. Viene con valor en la plantilla
POSTGRES_USER	Sí	Usuario del motor. Viene con valor en la plantilla
POSTGRES_PASSWORD	Sí, vacía	Contraseña del motor. Sin ella el contenedor de la base no arranca
JWT_SIGNING_KEY	Sí, vacía	Firma de los testigos de sesión. Mínimo 32 caracteres, sin valor por omisión
ADMIN_USERNAME	Sí	Nombre del primer administrador. La plantilla propone <code>admin</code>
ADMIN_PASSWORD	Sí, vacía	Su contraseña. Sin ella no se crea el administrador y no hay forma de entrar
PORTAL_PORT	No	Puerto publicado del portal. Por omisión, 8080
SWAGGER_ENABLED	No	Monta o retira la documentación ejecutable. Por omisión, montada
DEMO_SELLER_PASSWORD	No, vacía	Contraseña del vendedor de demostración que crea <code>seed.py</code> . Es la cuarta clave sin valor de la plantilla: sin ella el sistema arranca, y el que se detiene —diciéndolo— es el sembrador

De las nueve claves declaradas, **cuatro llegan vacías**: las tres obligatorias y la del vendedor de demostración. El archivo `.env` real está fuera del control de versiones: la plantilla declara las claves, nunca los valores.

5.2 Cómo llegan al servicio

El archivo de composición traduce cada clave del entorno a la variable que el servicio espera. Conocer esa correspondencia es lo que permite diagnosticar una configuración que no surte efecto.

Tabla 6. De la clave del entorno a la configuración del servicio

Clave en <code>.env</code>	Variable del contenedor	Clave de configuración
POSTGRES_*	ConnectionStrings__Postgres	ConnectionStrings:Postgres
JWT_SIGNING_KEY	Jwt__SigningKey	Jwt:SigningKey
ADMIN_USERNAME	Bootstrap__AdminUsername	Bootstrap:AdminUsername
ADMIN_PASSWORD	Bootstrap__AdminPassword	Bootstrap:AdminPassword
PORTAL_PORT	Cors__Origins__0	Cors:Origins:0
SWAGGER_ENABLED	Swagger__Enabled	Swagger:Enabled

Clave en .env	Variable del contenedor	Clave de configuración
— fijo en el compose —	Storage__RootPath	/var/lib/simple-stock-flow/media
— fijo en el compose —	Storage__PublicBaseUrl	/media

El doble guion bajo es la forma en que .NET representa la jerarquía de configuración en una variable de entorno.

5.3 Encender y apagar la documentación ejecutable

La página de Swagger está montada por omisión, **también en producción**, porque la interfaz de programación navegable es parte de lo que este sistema entrega. Para retirarla basta una línea en .env:

```
SWAGGER_ENABLED=false
```

Retirla no cambia nada más: ningún endpoint aparece ni desaparece con esa clave.

5.4 Cambiar el puerto publicado

Modifique PORTAL_PORT en .env y vuelva a levantar. Tenga en cuenta que esa misma clave alimenta el origen permitido de la política de origen cruzado del servicio, de modo que ambos se mueven juntos y no hay que tocar dos sitios.

6. Operación

6.1 Arrancar, parar y reiniciar

```
docker compose up -d --wait      # arrancar y esperar a que estén sanos
docker compose stop              # parar sin borrar nada
docker compose restart service   # reiniciar un solo contenedor
docker compose down              # bajar, conservando los volúmenes
```

Cuidado. `docker compose down -v` **borra los volúmenes**, y con ellos la base de datos y todas las imágenes de producto subidas. No es recuperable sin un respaldo.

6.2 Ver el estado y la salud

```
docker compose ps                # estado y salud de los tres
curl -i http://localhost:8080/    # el portal responde
curl -i http://localhost:5000/health # el proceso responde (solo en desarrollo)
```

Nota. `/health` dice que **el proceso está vivo**, no que sus dependencias lo estén: no consulta la base de datos. Para saber si la base responde, mire la salud del contenedor `db`, que sí ejecuta `pg_isready`.

6.3 Leer los registros

El servicio escribe en **JSON compacto**, una línea por evento, con identificador de correlación. Está pensado para que lo consuma una herramienta, no para leerlo a simple vista.

```
docker compose logs -f service    # seguir en vivo
docker compose logs service --tail 50 # las últimas 50 líneas
docker compose logs service | grep '"StatusCode":500' # solo los errores
```

Tabla 7. Campos útiles de cada línea de registro

Campo	Qué contiene
@t	Instante en UTC
@mt	La plantilla del mensaje
@tr / @sp	Identificadores de traza y de tramo, para seguir una petición completa. <code>verify.sh</code> §6c comprueba que ninguna línea de petición se queda sin @tr
RequestMethod / RequestPath	Método y ruta de la petición
StatusCode	Código con el que se respondió
Elapsed	Milisegundos que tardó
RequestId	Identificador de la petición dentro de la conexión

6.4 Inspeccionar la base de datos

Las tablas viven en el esquema sales. El historial de migraciones **no**: está donde lo pone el mapeador, en public."__EFMigrationsHistory".

```
docker compose exec db psql -U simple_stock_flow -d simple_stock_flow

# las tablas del sistema
docker compose exec db psql -U simple_stock_flow -d simple_stock_flow -c '\dt sales.*'

# qué migraciones se han aplicado
docker compose exec db psql -U simple_stock_flow -d simple_stock_flow \
  -c 'select "MigrationId" from public."__EFMigrationsHistory";'
```

6.4.1 Qué garantiza el esquema

Dos cosas conviene saber antes de tocar la base a mano, porque el motor ya no deja pasar lo que antes toleraba:

- **La venta apunta a su autor con integridad referencial.** sale.sold_by_user_id es clave foránea hacia sales."user" en modo RESTRICT: borrar un usuario que registró ventas **falla**, en lugar de dejar las ventas apuntando al vacío. Tiene su propio índice.
- **La línea de venta congela la categoría.** sale_item.category_name es NOT NULL y sin valor por omisión, de modo que una etiqueta olvidada rompe en voz alta en vez de colarse vacía.

Los índices del catálogo son **parciales**: solo cubren los productos activos (WHERE deleted_at IS NULL). Uno ordena por categoría y nombre; el otro es de trigramas sobre el nombre, para buscar por partes de palabra.

6.5 Respaldo

El estado del sistema vive en **dos sitios**, y un respaldo que solo cubra uno de los dos no sirve: la base de datos y el volumen de binarios. La base guarda la clave del archivo; los bytes están en el volumen.

6.5.1 La base de datos

```
docker compose exec -T db pg_dump -U simple_stock_flow -d simple_stock_flow \
  --format=custom > respaldo-$(date +%F).dump
```

6.5.2 El volumen de imágenes

```
MSYS_NO_PATHCONV=1 docker run --rm \
  -v simple-stock-flow_media:/origen:ro -v "$PWD":/destino \
  postgres:16-alpine tar czf /destino/media-$(date +%F).tgz -C /origen .
```

Nota. Se reutiliza la imagen postgres:16-alpine como herramienta porque **ya está descargada**: así el respaldo no depende de la red. El prefijo MSYS_NO_PATHCONV=1 es obligatorio en Git Bash sobre Windows; sin él, el shell traduce /destino a una ruta de Windows y el comando falla con un error que no menciona la causa.

Los dos procedimientos anteriores se ejecutaron contra este sistema y produjeron un volcado válido y un archivo comprimido con los binarios.

6.6 Restauración

```
# base de datos
docker compose exec -T db pg_restore -U simple_stock_flow -d simple_stock_flow \
  --clean --if-exists < respaldo-2026-09-20.dump

# volumen de imágenes
MSYS_NO_PATHCONV=1 docker run --rm \
  -v simple-stock-flow_media:/destino -v "$PWD":/origen \
  postgres:16-alpine tar xzf /origen/media-2026-09-20.tgz -C /destino
```

Cuidado. La restauración **sobrescribe** lo que haya. A diferencia de los comandos de respaldo, este procedimiento no se ejecutó durante la redacción de este manual, precisamente porque habría destruido los datos del entorno. Ensáyelo en un entorno desechable antes de necesitarlo de verdad.

6.7 Desplegar una versión nueva del código

1. Respalde la base y el volumen de imágenes, como en el apartado anterior.
2. Actualice los directorios `simple-stock-flow-api` y `simple-stock-flow-portal` con el código nuevo.
3. Reconstruya y levante: `docker compose up -d --build --wait`.
4. Compruebe que los tres contenedores vuelven a `healthy` y que el portal responde.

Atención. El indicador `--build` no es opcional en la práctica. Sin él, `--wait` **adopta la imagen que ya lleva la etiqueta** y el sistema sigue sirviendo el paquete anterior mientras todas las comprobaciones pasan en verde. Las migraciones pendientes se aplican solas al arrancar el servicio; no hay paso manual.

7. La interfaz de programación

7.1 Cómo se obtiene un testigo

Todas las operaciones salvo tres exigen sesión. La sesión se abre contra el extremo de autenticación y devuelve un testigo firmado que viaja en la cabecera `Authorization` con el prefijo `Bearer`.

1. Envíe usuario y contraseña a `POST /api/auth/login`.
2. Tome el campo `accessToken` de la respuesta.
3. Añádalo a cada petición como `Authorization: Bearer <testigo>`. En el portal lo hace un interceptor y ningún caso de uso se entera.
4. En Swagger, pulse **Authorize** y pegue solo el testigo: la página añade el prefijo por usted.

Atención. El sistema es **monomonedado**: todo importe es en pesos colombianos. No convierte. Si un precio llega en otra moneda, el dominio **lanza y lo dice** en vez de quedarse con el número y tirar la moneda, que es lo que hacía antes y lo que convertía una cifra en dólares en una cifra en pesos sin que nadie se enterara.

7.2 Inventario de operaciones

La forma exacta de cada petición y cada respuesta está en Swagger, generada del propio código; aquí está **qué hay y quién puede llamarlo**, que es lo que se necesita para operar.

Tabla 8. Las quince operaciones y su autorización

Método	Ruta	Quién puede	Para qué
POST	<code>/api/auth/login</code>	Anónimo	Abrir sesión y obtener el testigo
POST	<code>/api/auth/register</code>	Administrador	Crear un operador
GET	<code>/api/categories</code>	Autenticado	Las categorías, para el selector del formulario
GET	<code>/api/products</code>	Autenticado	El catálogo, paginado y filtrable
POST	<code>/api/products</code>	Administrador	Crear un producto
GET	<code>/api/products/{id}</code>	Autenticado	Un producto por identificador
PUT	<code>/api/products/{id}</code>	Administrador	Reemplazar un producto
DELETE	<code>/api/products/{id}</code>	Administrador	Retirar un producto (baja lógica)
POST	<code>/api/products/{id}/image</code>	Administrador	Adjuntar la imagen del producto
POST	<code>/api/sales</code>	Autenticado	Registrar una venta y descontar existencias
GET	<code>/api/sales</code>	Autenticado	Las ventas de un periodo, paginadas

Método	Ruta	Quién puede	Para qué
GET	/api/sales/{id}	Autenticado	Una venta con sus líneas
GET	/api/reports/sales	Autenticado	El reporte agregado de un periodo
GET	/health	Anónimo	Señal de vida del proceso
GET	/media/{clave}	Anónimo	Los bytes de una imagen almacenada

Una venta registrada **no se modifica ni se borra**: no existe PUT ni DELETE sobre ventas, y es deliberado.

7.3 Swagger como consola de pruebas

La página no solo documenta: ejecuta. Con el testigo pegado en **Authorize**, cada operación se puede lanzar desde el navegador contra el sistema real, lo que la convierte en la herramienta de diagnóstico más rápida que tiene a mano.

7.4 Qué significa cada código de respuesta

Toda excepción de negocio se traduce a un código HTTP **en un solo punto** del sistema. Esa correspondencia es estable y merece conocerse de memoria para diagnosticar sin leer código.

Tabla 9. Códigos de respuesta y su causa

Código	Qué ocurrió	Qué hacer
200 / 201	La operación salió bien	Nada
400	La petición no cumple la forma declarada. El cuerpo trae title y detail en español , un errors con los campos en camelCase y un traceId	Leer detail: nombra los campos. El traceId es el hilo hasta la línea del registro
401	No hay testigo, o no es válido	Volver a abrir sesión
403	El testigo es válido pero el rol no basta	La operación exige administrador
404	No existe nada con ese identificador	Comprobar el identificador
409	Conflicto de concurrencia tras agotar los reintentos	Reintentar la operación: otra la ganó
422	La petición estaba bien formada, pero la regla de negocio falló	Leer el mensaje: dice qué regla, en español

El 422 es el caso que más se confunde con un error de forma. Si aparece, la petición estaba bien formada y lo que falló fue el negocio.

Nota. Ninguna operación prevista devuelve 500. Lo que antes reventaba —un nombre de producto por encima de los 200 caracteres— lo rechaza ahora la validación con un 400 que dice qué campo sobra. Si ve un 500, no es un caso previsto: es un defecto, y conviene abrirlo con su traceId.

7.5 La operación de venta, paso a paso

Registrar una venta es la operación donde se concentra el riesgo del sistema, y por eso es la que conviene tener en la cabeza antes de diagnosticar un 409 o un 422. La figura siguiente la recorre entera: el caso de uso ordena los pasos, el dominio decide, y la escritura queda condicionada a que las filas leídas no hayan cambiado.

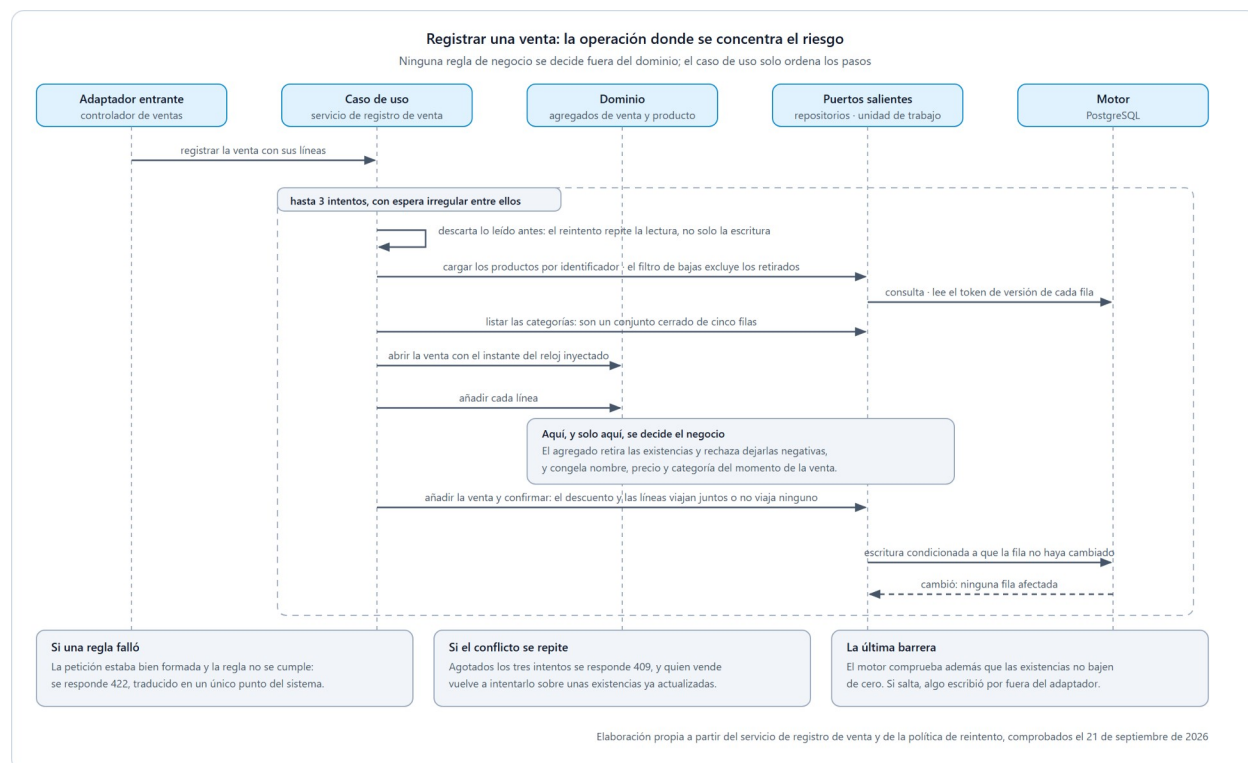


Figura 4. El registro de una venta: qué decide el dominio, qué congela la línea y cuándo se reintenta

- **El descuento de existencias y las líneas viajan juntos o no viaja ninguno.** No hay un estado intermedio en el que la venta exista y el stock no haya bajado.
- **La línea congela el nombre, el precio y la categoría del momento de la venta.** Es lo que permite que un reporte de un periodo cerrado siga diciendo lo mismo cuando el catálogo cambia.
- **El reintento repite la lectura, no solo la escritura.** Hasta tres intentos con espera irregular entre ellos; agotados los tres, la operación responde 409 y quien la lanzó debe repetirla.

7.6 La ruta pública de imágenes

`/media/{clave}` **no es un extremo enrutado**: lo sirve el middleware de archivos estáticos sobre el volumen de binarios. Al no estar enrutado, no atraviesa la autorización, y no hay forma de protegerlo sin cambiar el mecanismo. Es lo que permite que la etiqueta de imagen del portal funcione sin cabeceras.

Las claves son opacas —32 caracteres hexadecimales sin relación con el nombre original—, de modo que no se pueden adivinar ni enumerar, pero **una dirección filtrada sirve la imagen a cualquiera de forma**

indefinida. Es un límite conocido y aceptado: proteger esa ruta exigiría cambiar el mecanismo que la sirve.

```
docker compose exec service ls -la /var/lib/simple-stock-flow/media
```

7.7 El camino que recorre una petición

Saber por dónde pasa una petición es lo que permite decidir **dónde mirar** cuando algo falla: si el portal responde pero la operación no, el problema está después del proxy; si ni siquiera hay registro en el servicio, está antes.

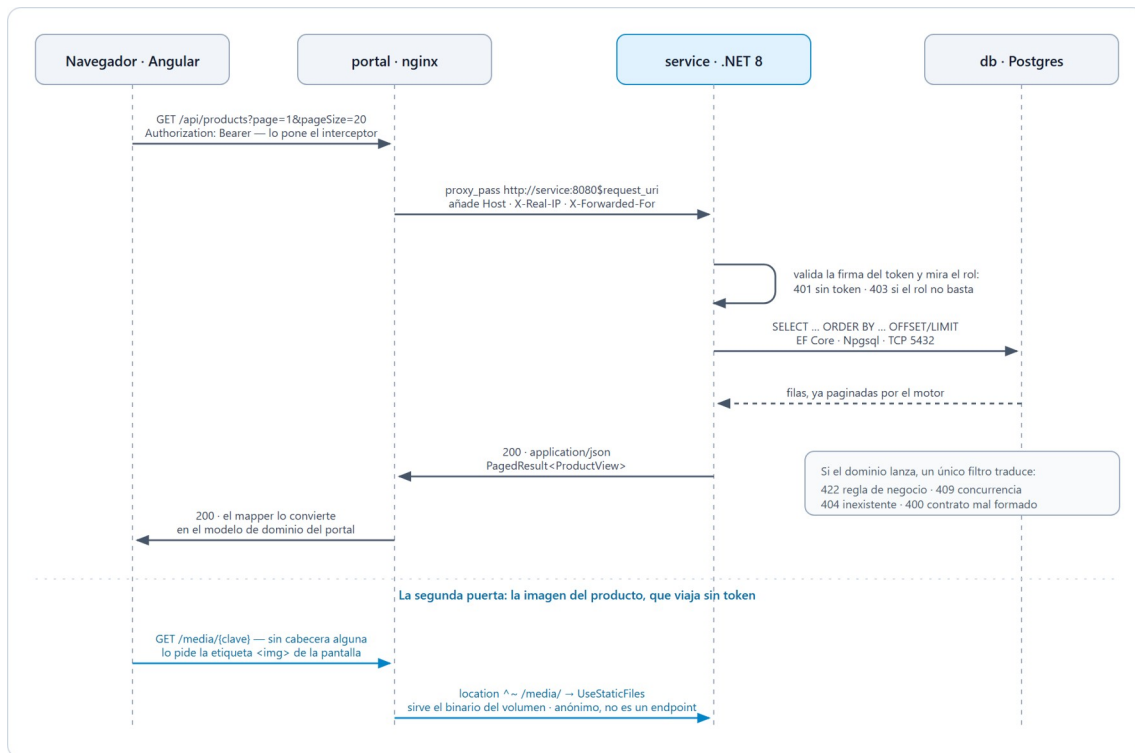


Figura 5. Camino completo de una petición, de la pantalla al motor y de vuelta

Dos detalles del intermediario importan cuando se despliega detrás de él, y los dos se arreglaron por lo mismo: una location de prefijo pierde contra una expresión regular, y una cabecera Host mal elegida pierde el puerto.

- **Los tres bloques que van a la interfaz de programación —/api/, /media/ y /swagger— llevan ^~, que es lo único que les gana a la expresión regular de recursos estáticos.** Sin eso, cualquier ruta terminada en .jpg, .png, .css o .js la atrapaba el bloque de estáticos y respondía con la página de error del propio intermediario.
- **Los tres reenvían Host: \$http_host, no \$host.** La diferencia es el puerto: \$host lo descarta, y una cabecera Location de un 201 salía apuntando al puerto 80, donde no hay nada escuchando.

Nota. Se comprueba sin abrir el archivo: `curl -o /dev/null -w "%{http_code} %{size_download}"`

`http://localhost:8080/api/loquesea.jpg` debe responder **404 con cero bytes** —el cuerpo vacío de la interfaz de programación—. Un 404 de unos 153 bytes sería la página HTML del intermediario, y significaría que la regla volvió a perder.

8. Mantenimiento del código

Los bloques de este capítulo empiezan con un `cd ..` hacia el repositorio que toca, igual que los del anexo. Así funcionan tanto si viene de `simple-stock-flow-infra` como si acaba de ejecutar el bloque anterior: los directorios son hermanos, de modo que `..` es siempre la carpeta que los contiene.

8.1 Compilar y probar el servicio

```
cd ../simple-stock-flow-api
dotnet build SimpleStockFlow.sln
dotnet test SimpleStockFlow.sln
```

Atención. La solución compila con **toda advertencia tratada como error**. Una advertencia nueva rompe la construcción, y es intencionado. Las pruebas de integración necesitan Docker en marcha: levantan un PostgreSQL real en un contenedor efímero.

8.2 Compilar y probar el portal

```
cd ../simple-stock-flow-portal
npm ci
npx ng test --watch=false --browsers=ChromeHeadless
npm run build -- --configuration=production
```

Para trabajar con recarga en vivo contra la interfaz de programación real, levante el sistema con el solapamiento de desarrollo y ejecute `npm start`: la configuración de proxy del proyecto reenvía `/api` y `/media` a `http://localhost:5000`.

8.3 La regla que no se puede romper

El sistema está organizado en capas, y las dependencias apuntan **siempre hacia dentro**. Lo que convierte esa regla en algo comprobable es que cada capa es un proyecto independiente: un adaptador no puede llamar a otro adaptador porque no lo referencia, y el dominio no puede hablar del mapeador porque no lo conoce. La impone el compilador, no la disciplina.

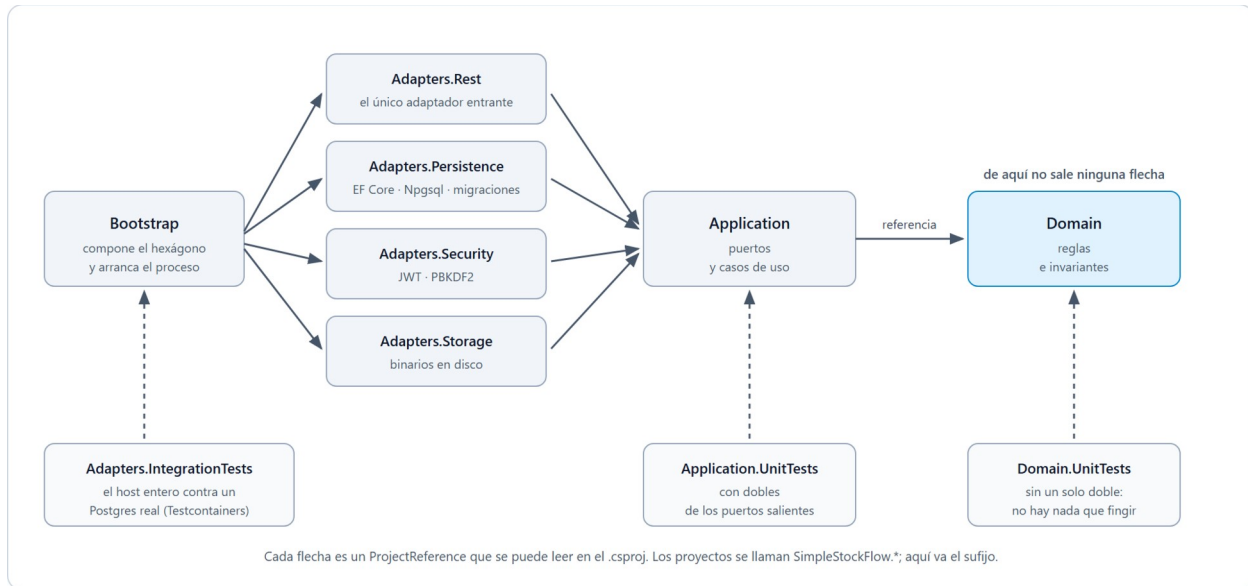


Figura 6. Referencias reales entre los proyectos de la solución

Nota. Existe un único archivo en cada repositorio donde un puerto encuentra su adaptador: `bootstrap/Composition/PortBindings.cs` en el servicio e `infrastructure/providers.ts` en el portal. Si aparece una construcción directa de infraestructura fuera de ahí, la organización está rota aunque todo compile.

8.4 Dónde va cada cosa

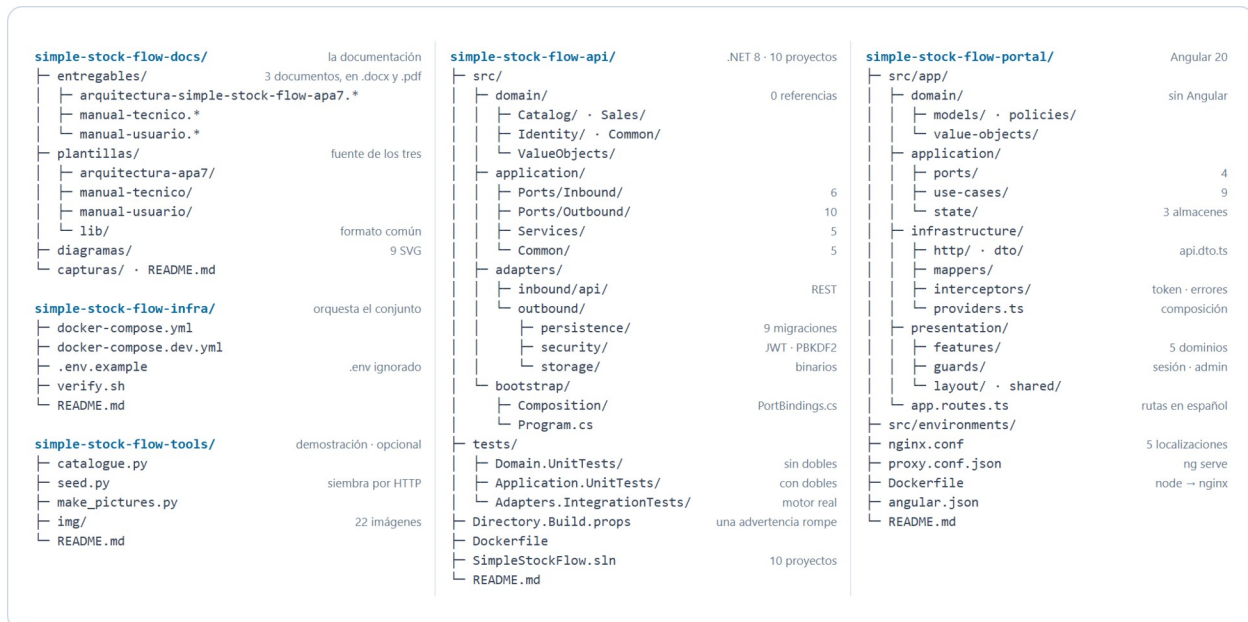


Figura 7. Estructura interna de los repositorios, con el sembrador ya fuera de la infraestructura

Los dos repositorios de código repiten la misma forma, y esa simetría es lo que permite probar la lógica del portal sin levantar un servidor y la del servicio sin abrir un navegador.

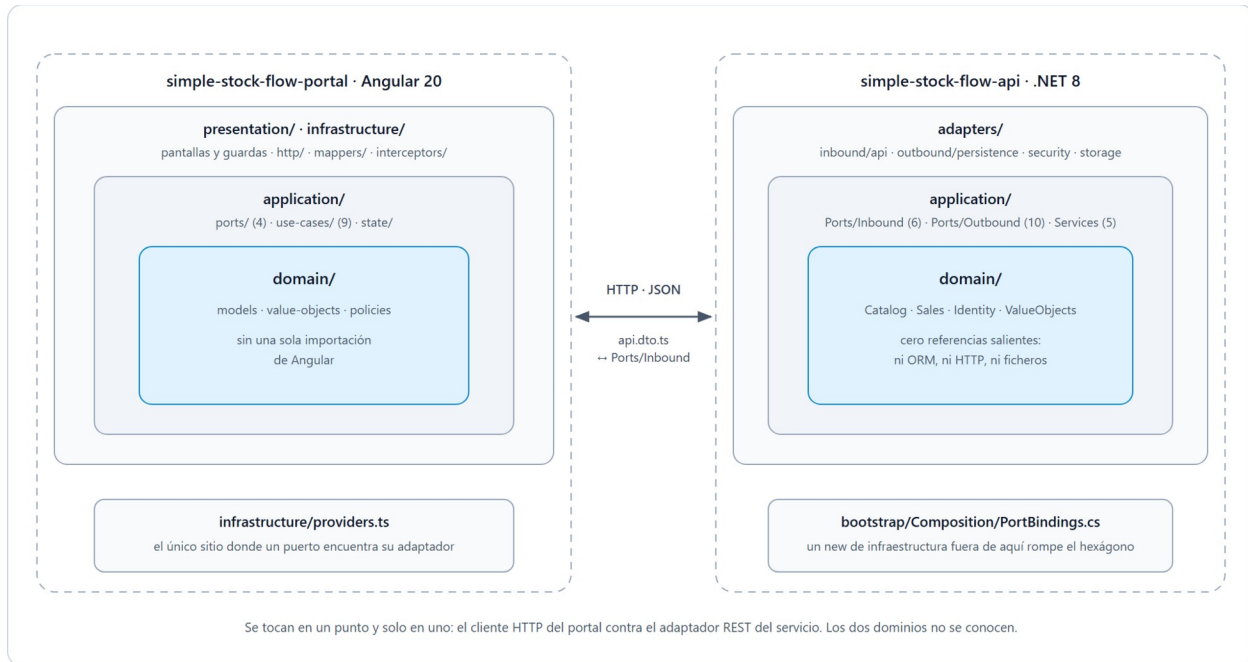


Figura 8. Las mismas tres capas a los dos lados, y el único punto donde se tocan

8.5 Añadir un caso de uso

1. Escriba primero la prueba en rojo. Es una regla del proyecto, no una recomendación.
2. Declare el método en el **puerto entrante** que corresponda, en `src/application/Ports/Inbound/`. Si el caso de uso necesita algo de fuera, declare también el **puerto saliente** en `Ports/Outbound/`, siempre en el lenguaje del dominio y sin mencionar ninguna tecnología.
3. Impleméntelo en el servicio de aplicación correspondiente, en `src/application/Services/`.
4. Implemente el puerto saliente nuevo en el adaptador que toque, bajo `src/adapters/outbound/`.
5. Registre la pareja puerto–adaptador en `PortBindings.cs`, que es el único sitio donde se hace.
6. Exponga el caso de uso en el controlador de `src/adapters/inbound/api/Controllers/`, documentando la operación: Swagger lee sus descripciones de los comentarios XML de ese proyecto y de ningún otro sitio.

8.6 Añadir una migración

El esquema pertenece al servicio y se modifica con migraciones, nunca con SQL suelto contra la base.

```
cd ../simple-stock-flow-api
Jwt__SigningKey="una-clave-de-al-menos-32-caracteres-aqui" \
dotnet ef migrations add <Nombre> \
  --project src/adapters/outbound/persistence \
  --startup-project src/bootstrap \
  --output-dir Migrations
```

Nota. La clave de firma hace falta **para crear la migración**, no solo para ejecutar: `dotnet ef` construye el host real de la aplicación, y ese host se niega a arrancar sin ella. La migración se aplica sola en el siguiente arranque del servicio.

9. Diagnóstico de problemas

9.1 Síntomas frecuentes

Tabla 10. Qué hacer cuando algo falla

Síntoma	Causa habitual	Remedio
El servicio no arranca y se reinicia solo	Falta JWT_SIGNING_KEY o mide menos de 32 caracteres	Mire <code>docker compose logs service</code> : el mensaje lo dice con todas las letras. Corrija <code>.env</code> y vuelva a levantar
El portal responde 502 después de reconstruir el servicio	El proxy resolvió la dirección del contenedor anterior	Está previsto: la configuración vuelve a resolver cada 10 segundos. Si persiste, <code>docker compose restart portal</code>
<code>http://localhost:5000</code> deja de responder de golpe	Se levantó con el archivo base una pila que corría con el solapamiento	Vuelva a levantar con los dos archivos
Una ruta de la API terminada en <code>.jpg</code> o <code>.css</code> devuelve una página HTML	El bloque de estáticos del intermediario le ganó a la regla de prefijo	No debería ocurrir: los tres bloques llevan <code>^~</code> . Si ocurre, alguien editó <code>nginx.conf</code> . Compruébelo con la sonda del capítulo 7
Todo responde 401	No hay testigo, expiró, o no se envía la cabecera	Vuelva a abrir sesión. Compruebe que la cabecera viaja con el prefijo Bearer
Una operación responde 403 con sesión válida	El rol no basta: la operación exige administrador	Entre con una cuenta de administrador
Una venta responde 409	Dos ventas compitieron por las mismas existencias y se agotaron los reintentos	Reintente. Es el comportamiento correcto, no un fallo
Una operación responde 422	Una regla de negocio la rechazó	Lea el mensaje: está en español y dice qué regla falló
El primer arranque tarda mucho	Se están construyendo las dos imágenes desde cero	Normal la primera vez. Las siguientes reutilizan la caché
El servicio arranca, migra y muere contra un motor administrado	El usuario de base no puede crear extensiones, y una migración instala <code>pg_trgm</code>	Conceda el permiso, o instale la extensión a mano antes de migrar

9.2 Cuando el documento y el sistema discrepan

Todo lo que este manual afirma se ejecutó contra el sistema real antes de escribirlo, y las afirmaciones que pueden envejecer llevan su fecha. Aun así, un documento describe el sistema de un día concreto y el sistema cambia. **Si lo que usted observa contradice lo que aquí se lee, gana lo que observa.**

Tres comprobaciones bastan para saber de qué lado está la verdad, y ninguna necesita leer código: `docker compose ps` dice si los tres contenedores están sanos, `./verify.sh` recorre el sistema entero y

termina contando cuántas comprobaciones pasaron, y la página de /swagger describe la interfaz de programación **generada del propio código**, de modo que no puede desfasarse de él.

Nota. Antes de dar por nuevo un comportamiento extraño, compruebe si es intencionado: el 409 de una venta, el 422 de una regla de negocio y el 403 de un rol insuficiente son respuestas correctas del sistema, y las tres aparecen en la tabla anterior con esa etiqueta.

10. Anexo: comandos de referencia rápida

Todos se ejecutan desde `simple-stock-flow-infra` salvo donde se indique.

Tabla 11. Referencia rápida

Para	Orden
Levantar en producción	<code>docker compose up -d --wait</code>
Levantar en desarrollo	<code>docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d --wait</code>
Ver el estado	<code>docker compose ps</code>
Seguir los registros del servicio	<code>docker compose logs -f service</code>
Entrar a la base de datos	<code>docker compose exec db psql -U simple_stock_flow -d simple_stock_flow</code>
Ver las migraciones aplicadas	<code>select "MigrationId" from public."__EFMigrationsHistory";</code>
Respalidar la base	<code>docker compose exec -T db pg_dump -U simple_stock_flow -d simple_stock_flow --format=custom > respaldo.dump</code>
Listar las imágenes almacenadas	<code>docker compose exec service ls -la /var/lib/simple-stock-flow/media</code>
Verificación completa	<code>./verify.sh</code>
Sembrar la demostración	<code>cd ../simple-stock-flow-tools && python seed.py</code>
Reconstruir y desplegar	<code>docker compose up -d --build --wait</code>
Bajar conservando los datos	<code>docker compose down</code>
Compilar y probar el servicio	<code>cd ../simple-stock-flow-api && dotnet test SimpleStockFlow.sln</code>
Probar el portal	<code>cd ../simple-stock-flow-portal && npx ng test --watch=false --browsers=ChromeHeadless</code>

11. Documentos relacionados

Tabla 12. Dónde seguir leyendo

Documento	Qué responde
Documento de arquitectura	Por qué el sistema es como es: los artefactos, las cuatro decisiones con sus alternativas descartadas, los atributos de calidad con su precio y el registro de riesgos
Manual de usuario	Cómo se opera el sistema desde la pantalla: catálogo, ventas y reporte
Página de documentación ejecutable, en /swagger	El cuerpo exacto de cada petición y de cada respuesta, generado del propio código
simple-stock-flow-infra/README.md	Cómo se levanta el conjunto y dónde están los datos
simple-stock-flow-infra/.env.example	Todas las claves de configuración, y ningún valor secreto
simple-stock-flow-tools/README.md	El sembrador y las imágenes de demostración: qué instala y cómo se ejecuta
simple-stock-flow-api/README.md	La solución, sus diez proyectos y cómo se compila y se prueba
simple-stock-flow-portal/README.md	El portal, sus capas y cómo se ejecutan sus pruebas

Los dos primeros acompañan a este manual; los seis restantes viajan dentro del sistema y se actualizan con él.